Lecture 3: Nonparametric regression, additive models, classification?

Max G'Sell
Machine Learning II: 46-927

February 4, 2019

Announcements

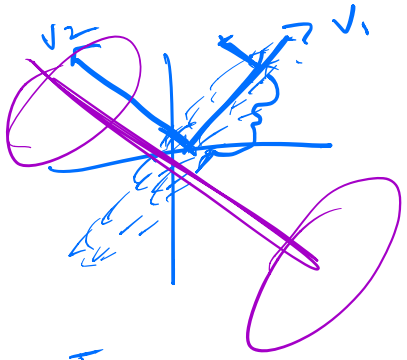
- Homework 2 is due today. Homework 3 comes out tomorrow night and is due on February 10.
- `rpy2` is turning out to be a pain for some. It's ok to compile R directly as an Rmd document if you prefer. Even better to merge the PDFs if you can. We will need R for a few more topics as we go along.

Today

Today will focus mostly on nonparametric regression approaches and extensions.

- First, a few comments on homework, including a mistake
- Splines, smoothing splines
- Additive models
- Time permitting: Classification

$$\hat{y} = UU^T y$$



$$X = UDV^T$$

$$X^TX = VDU^TUDV^T$$

$$= VD^2V^T$$

$$XV = UD$$

$$\hat{y}_R = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda} (u_j^T y) u_j$$

$$(X^TX)$$

$$U \Lambda U^T$$

$$\begin{pmatrix} d_1^2 & & \\ & d_2^2 & \\ & & \ddots \end{pmatrix}$$

Review: What next?

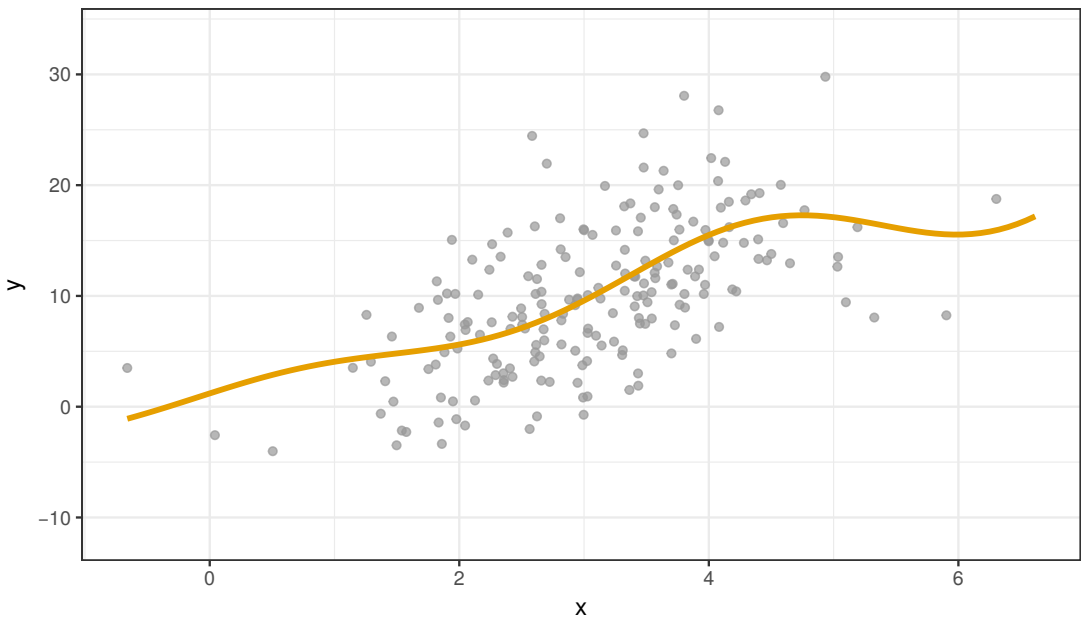
We started out by noting that we want to approximate $\mathbb{E}(Y|X = x)$.

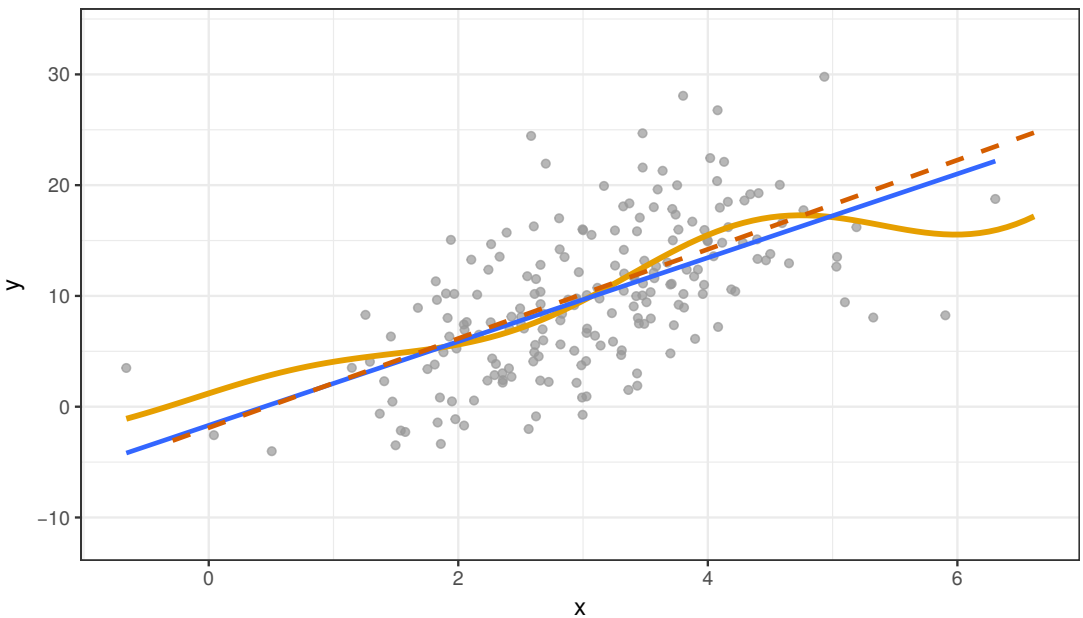
We thought about linear approximations, $\mathbb{E}(Y|X = x) = x^T\beta$, and showed that we can beat the least squares estimate with regularization to

- Reduce the variance (at the cost of bias).
- Improve interpretability by adding sparsity

However, at the end of the day, we're still using something linear!

How do we move to something with lower bias instead of lower variance?


30
20
10
0
−10
y
0
2
4
6
x


30
20
10
0
−10
y
0
2
4
6
x

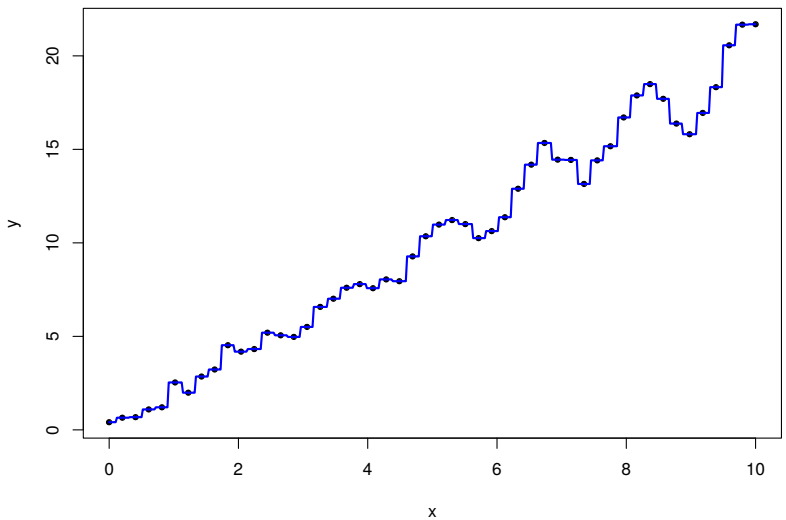
Nonparametric regression

We will talk about two overall approaches to nonlinear regression. We begin with $X \in \mathbb{R}$, the one dimensional problem shown in the pictures above.

Kernel-based methods: We use a version of local averaging to approximate the function. Includes kernel smoothing, local linear regression. Closely related to k-nearest neighbors.

Spline-based methods. We fit a flexible piecewise polynomial to the data in a way that compromises between goodness-of-fit and smoothness.

k-nearest neighbors



Recall that we want $\hat{\mu}(x)$ to be a good approximation of $\mathbb{E}(Y|X = x)$. What if we don't want to assume a strong form for the $\mathbb{E}(Y|X = x)$ function?

We might still be willing to assume that points near x are similar to x. We can use this to estimate the conditional mean!

Kernel regression

$$\hat{\mu}(x) = \sum_{i=1}^{n} y_i \frac{K(x_i, x)}{\sum_{j=1}^{n} K(x_j, x)}$$

$$K(x, x_j) = \frac{1}{\sqrt{2\pi h}} e^{-(x-x_j)^2/2h} \qquad \text{(Gaussian, example)}$$

Kernel regression gives a very simple way to get non-linear estimates of a function.

As we tune h, we determine how locally the fit looks at the training points, trading off bias and variance!

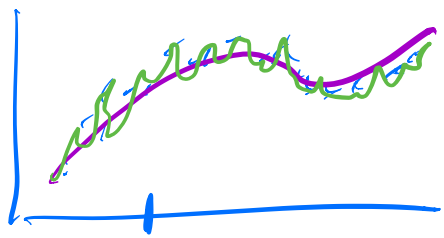
For nice functions, can converge at $O\left(\frac{1}{n^{4/(p+4)}}\right)$.

What does it look like to evaluate?

An alternative approach: smoothing splines

Instead of kernel smoothing, suppose we started by trying to build a flexible function that:

- Approximates our data well.
- Is smooth.



We could try to encode this as an optimization problem:

$$\hat{f} = \underset{f}{\operatorname{argmin}} \quad \sum_{i=1}^{n} \left(y_i - f(x_i) \right)^2 + \lambda \int \left(f''(x) \right)^2 dx$$

over twice-differentiable f.

Smoothing splines

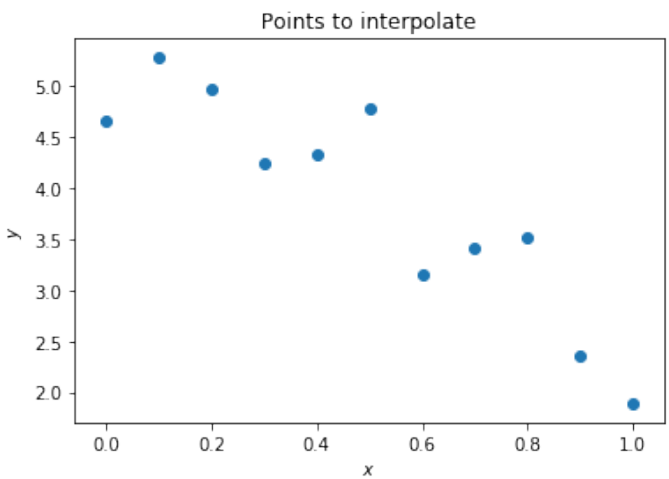
This is just another penalized regression problem!

$$\hat{f} = \underset{f}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$$

- As λ gets larger: f gets smoother.
- If $\lambda \to \infty$: f -> Least squares line
- As λ gets smaller: wiggly f
- As $\lambda \to 0$: f interpolates (x_i, y_i)

Digression: splines

Splines come out of function approximation and interpolation. Suppose that I wanted to represent a wiggly function, or just interpolate some points.



Kernel regression is very expensive to evaluate. A linear model isn't going to do a great job.

I could add some wiggle with a higher order polynomial...
[Switch to python interpolation notebook]

Splines

Cubic splines give a very nice solution to the interpolation problem.

The function between points is modeled as piecewise cubic. This gives well-behaved wiggle.

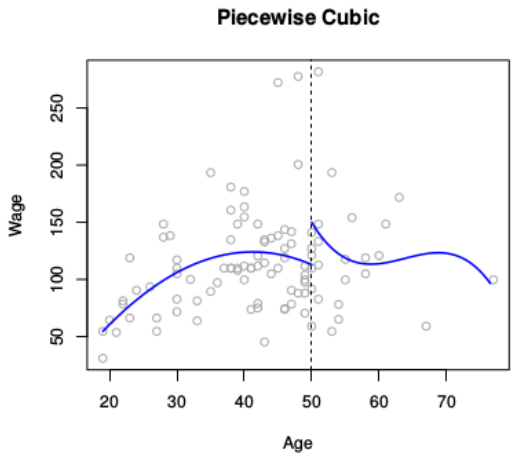
The function is constrainted to have continuous $f(x), f'(x), f''(x)$ at the points where cubics change.

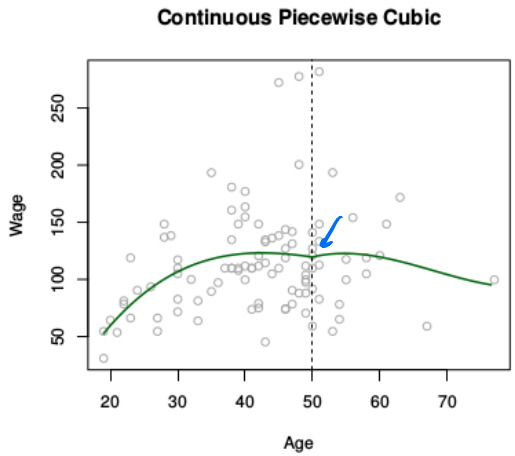
Despite the apparent complexity, we can write a K-knot spline as

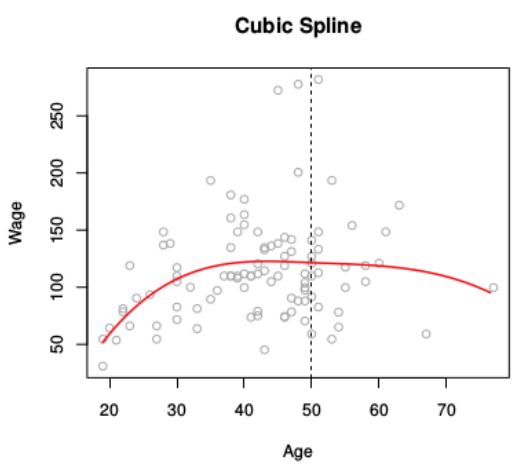
$$f(x) = \beta_0 + \beta_1 b_1(x) + \beta_2 b_2(x) + \cdots + \beta_{K+3} b_{K+3}(x)$$

for very well-behaved bases $b_1, \ldots, b_{K+3}$, making computation fast and efficient!

Splines


Piecewise Cubic
Wage
50
100
150
200
250
20
30
40
50
60
70
Age


Continuous Piecewise Cubic
Wage
50
100
150
200
250
20
30
40
50
60
70
Age


Cubic Spline
Wage
50
100
150
200
250
20
30
40
50
60
70
Age

Smoothing splines

Back to our problem:

$$\hat{f} = \underset{f}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$$

This looks like a pretty ugly optimization problem. However, the solution turns out to be very nice!

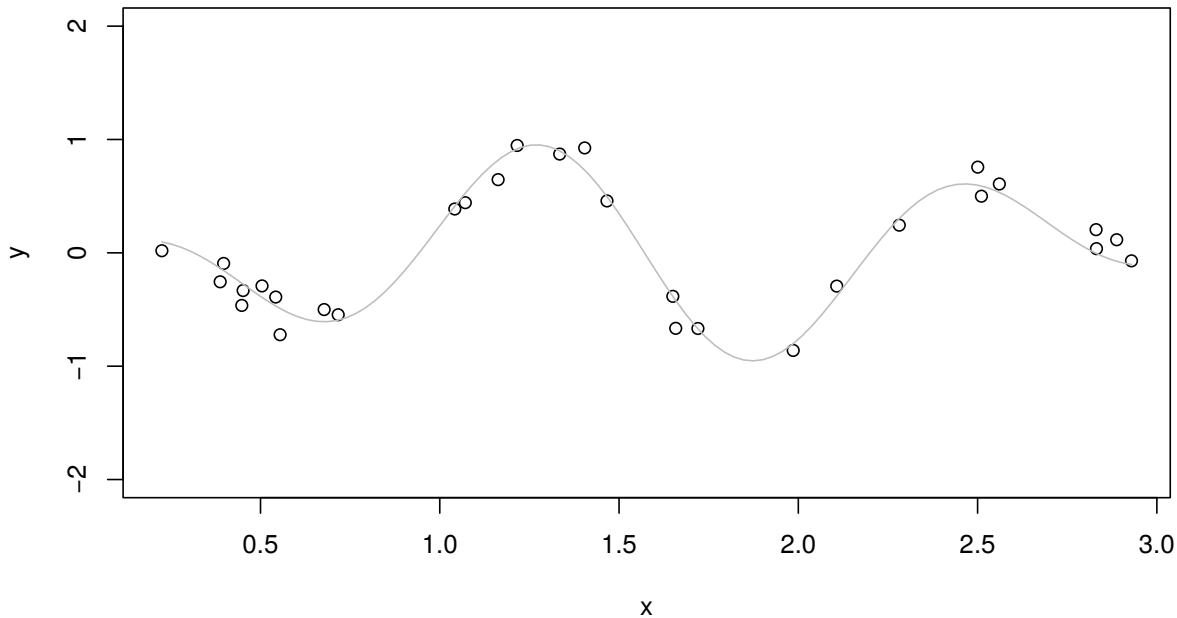
The minimizer is always a cubic spline!

$$\hat{Y} = S_\lambda Y$$

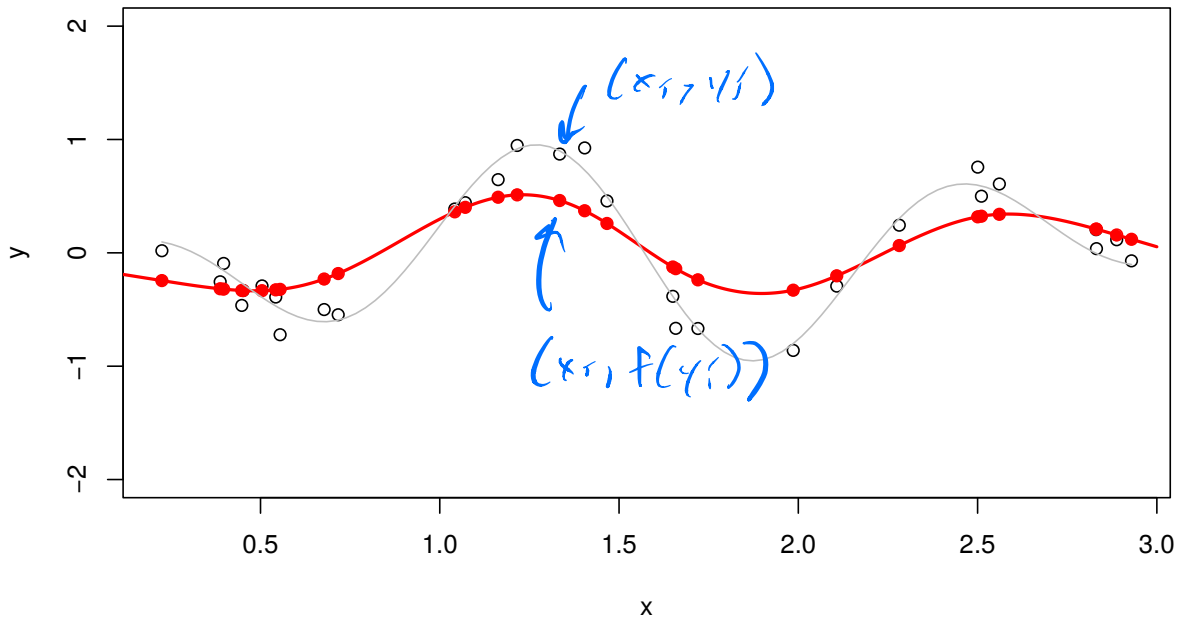
Furthermore, fitting the cubic spline is a linear operator on Y!

We can steal all the work done over the years in applied mathematics, making computation convenient and efficient.

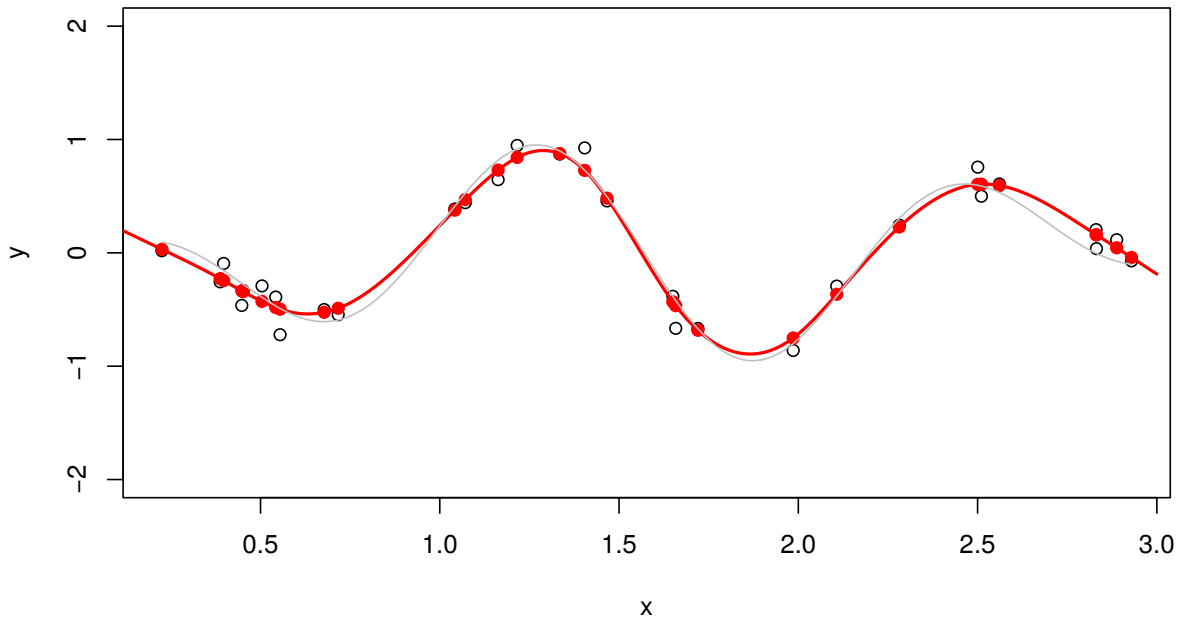
Smoothing splines



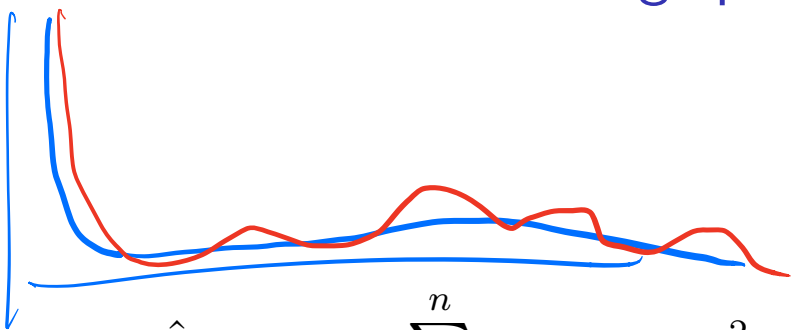
Smoothing splines



Smoothing splines



Smoothing splines



$$\hat{f} = \underset{f}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$$

This trick gives a computationally efficient way to swap smooth, flexible functions in for linear functions throughout our methods.

We'll see several applications of this in the course.

same smoothing everywhere

Fixed By: - Trend filtering.

- "Boosting."

Now what?

So far, we've seen that:

- We can build nonlinear function estimates that converge to the true regression function with enough data (and smoothness).
- We can make them efficient in one dimension, using splines.
- Our kernel methods break down in higher dimensions.

We don't usually just care about one-dimensional regression! What can we do?

More than one dimension

Suppose that we want to estimate Y from $X \in \mathbb{R}^p$, with $p > 1$.

For quite small p, we could still use kernel regression. There are also two generalizations of splines to low dimensions, but they suffer from the same curse.

We look for a compromise: more flexibility than linear regression, but less than general d-dimensional function estimation.

Additive models

Think about our linear models:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^{p} \hat{\beta}_j X_j$$

There are really two modeling choices going into this:

- Additivity: The each feature contributes additively to the prediction.
- Linearity: The values of each feature enter linearly.

It turns out that linearity can be relaxed without sacrificing much efficiency!

Additive models

Additive models:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^{p} \hat{f}_j(X_j)$$

We can choose the $\hat{f}_j$ in many different ways. We can even vary the choice for each feature!

- Linear function
- Polynomial function
- Smoothing spline
- Kernel regression
- Local linear fit

$$\min \sum_{i=1}^{n} (y_i - \hat{f}(x))^2 + \sum_{j=1}^{p} \lambda_j \int (\hat{f}_j''(x_j))^2 \, dx_j$$

In practice, fitting smoothing splines for each $\hat{f}_j$ is most common. This is what the `LinearGAM` function in python does (package `pygam`, and what `gam`, `mgcv` packages do in `R`. *(Note:* `mgcv` *is the easiest.)*

Additive models

To estimate a function linearly, we expect

$$MSE = O(1/n)$$

To estimate a 1-dimensional function nonparametrically, we expect

$$MSE = O(1/n^{4/5})$$

To estimate a d-dimensional function nonparametrically, we expect

$$MSE = O(1/n^{4/(d+4)})$$

Additive models

To estimate a d-dimensional function nonparametrically, we expect

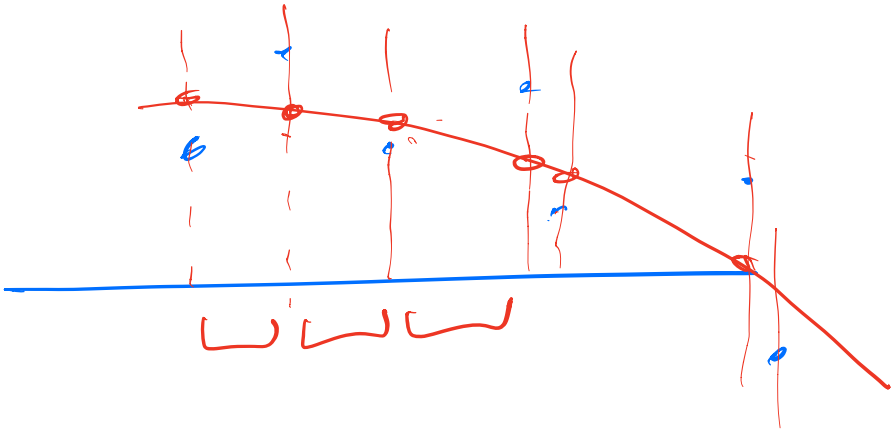
$$MSE = O\left(\frac{1}{n^{4/(d+4)}}\right)$$

To estimate a d-dimensional *additive* model, we just estimate each piece separately, giving

$$MSE = O\left(\frac{1}{n^{4/5}}\right)$$

This means that, if we don't care about this minor drop in convergence rate, we can replace linear regression with non-linear functions of each variable!

No more guessing at variable transformations by hand!



How about classification?

Recall that classification refers to problems where you see $X \in \mathbb{R}^p$, and try to guess $y \in \{1, \ldots, K\}$ using a classification function $f(X)$. This function is usually estimated from training data.

You may have seen several methods: Logistic regression, Generalized additive logistic models, Support Vector Machines (SVM), Nearest Neighbors, Naive Bayes, Decision Trees, Random Forests

We will mostly focus on two-class problems ($K = 2$), and we will often denote the classes as $\{0, 1\}$ or $\{-1, 1\}$.

Confusion matrix

- $Y = 1$ if an event of interest happened
- $Y = 0$ if it did not happen

In a classification problem, we can make $K(K-1)$ different kinds of mistakes!

This leads to the *confusion matrix*, which tabulates guessed categories against true categories.

In our 2×2 setting:

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Pos	False Positives
Guess $Y = 0$	False Neg.	True negatives

In general, there is one entry for each class in the rows and columns.

Confusion matrix: error rates

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

We are interested in many different quantities in the confusion matrix. Probably the most famous:

- Misclassification rate: $\frac{FP + FN}{n}$
- Accuracy: $1 - \hat{r} = \frac{TP + TN}{n}$

If False positives and False negatives are equally bad, then minimizing the misclassification rate is a good goal.

This is not always a good idea! We will come back to this later.

Cost/Loss for classification

When building classifiers, it is useful to think of the confusion matrix in terms of *losses*. This is analogous to loss functions in regression.

Because our guesses and outcomes are discrete, the loss function can be summarized by a matrix:

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	L_{11}	L_{01}
Guess $Y = 0$	L_{10}	L_{00}

Usually we think of $L_{11} = L_{00} = 0$.

In most cases, L_{01} (cost of guessing $Y = 1$ when it is actually $Y = 0$) is quite different from L_{10}.

However, the simplest loss, 0-1 loss, sets $L_{ij} = 1$ for $i \neq j$.

0-1 Loss

The 0-1 loss is unrealistic for many scenarios, since the cost for errors is usually asymmetric.

However, it gives us a good starting point for developing methods.

The methods that result from 0-1 loss can often be tweaked to give good results for other losses.

Suppose that we decide to minimize 0-1 loss. How should we carry it out?

Statistical decision theory

Ideally, we would choose our prediction $\hat{f}(X)$ to minimize the expected loss (also called expected prediction error or expected misclassification error).

$\{1, 2, \ldots, K\}$

$$\mathrm{EPE} = \mathbb{E}\left(\mathcal{L}(Y, \hat{f}(X))\right) = E\left[E\left(L(Y, f(X)) \mid X \right) \right]$$

$$= E\left[\sum_{k=1}^{K} L(k, f(X))\, P(Y = k \mid X) \right]$$

We conditioned on X, so that we can first take the expectation over Y with X held fixed, and then take the expectation over X.

Statistical decision theory

Now we minimize the inside of the expectation over $\hat{f}(X)$ for *each possible value* of X, pointwise:

$$\hat{f}(x) = \operatorname*{argmin}_{g \in \{1,\dots,K\}} \sum_{k=1}^{K} \mathcal{L}(k, g) \mathbb{P}(Y = k | X = x)$$

$$= \operatorname*{argmin}_{g} \sum_{k=1}^{K} \mathbb{1}_{g \neq k} \, P(Y = k | X = x)$$

$$= \operatorname*{argmin}_{g} \sum_{k=1}^{K} (1 - \mathbb{1}_{g = k}) \, P(Y = k | X = x)$$

$$= \operatorname*{argmin}_{g} \; 1 - \sum_{k=1}^{K} \mathbb{1}_{g = k} \, P(Y = k | X = x)$$

$$= \operatorname*{argmin}_{g} \; 1 - P(Y = g | X = x)$$

$$= \operatorname*{argmax}_{g} \; P(Y = g | X = x)$$

Statistical decision theory

The decision theory framework leads us to a natural rule for classification:

$$\hat{f}(x) = \underset{j=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(Y = j | X = x)$$

This predicts the most likely class, given the feature measurements $X = x \in \mathbb{R}^p$.

This is called the Bayes classifier, and it achieves the best possible expected misclassification error (if we could do it)

Statistical decision theory

Remember Bayes theorem, which says:

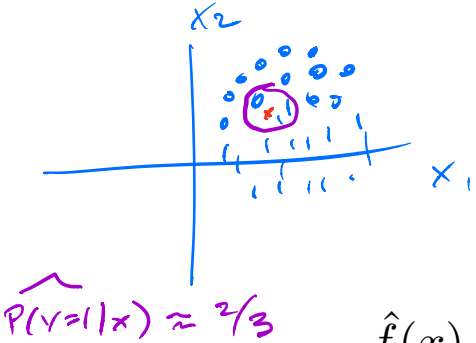
$$\mathrm{P}(Y = j|X = x) = \frac{P(X = x|Y = j)P(Y = j)}{P(X = x)}$$

[Note: $P(X = x|Y = j)$ and $P(X = x)$ could also be densities.]

Let $\pi_j = \mathrm{P}(Y = j)$ be the prior probability of class j.

Since the Bayes classifier compares the above quantity across $j = 1, \ldots K$ for $X = x$, the denominator is always the same, hence

$$\hat{f}(x) = \underset{j=1,\ldots K}{\operatorname{argmax}} \; \mathrm{P}(X = x|Y = j) \cdot \pi_j$$



Building a classifier

$$\hat{f}(x) = \underset{j=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(Y = j | X = x)$$

$$= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(X = x | Y = j) \cdot \pi_j$$

How do we actually use this to build a classifier?

1. We can try to estimate $\mathbb{P}(Y = j | X = x)$ directly.

   Logistic regression is a parametric version of this, while k-Nearest neighbors is a non-parametric version of this.

Building a classifier

$$\hat{f}(x) = \underset{j=1,\ldots K}{\operatorname{argmax}} \, \mathrm{P}(Y = j | X = x)$$
$$= \underset{j=1,\ldots K}{\operatorname{argmax}} \, \mathrm{P}(X = x | Y = j) \cdot \pi_j$$

How do we actually use this to build a classifier?

2. We could try to estimate the distributions $\mathbb{P}(X = x|Y = j)$ for each class. That is, given that an observation comes from class j, how are its covariates distributed?

   Linear discriminant analysis (LDA) and naive Bayes are both classic examples of this. We will come back to these later.

Logistic regression

How should we think about modeling $\mathbb{P}(Y = k|X = x)$ directly?

If we only had a few x values, we could directly look at Y conditional on each one:

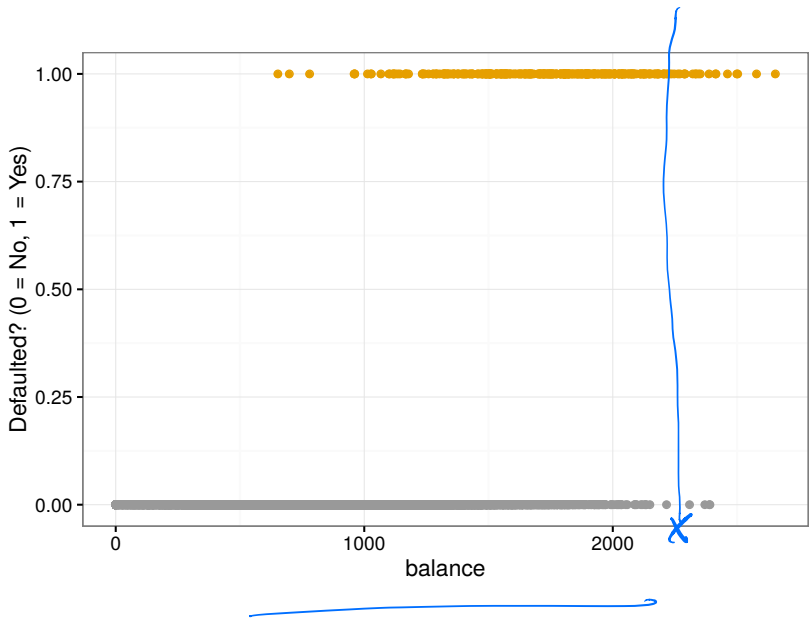
	Default=No	Default=Yes
Student=No	6850	206
Student=Yes	2817	127

$P(\text{Default}|\text{Student})$ $= \frac{127}{2817+127}$

$P(\text{Default}|\text{Not Student})$

Discriminative models

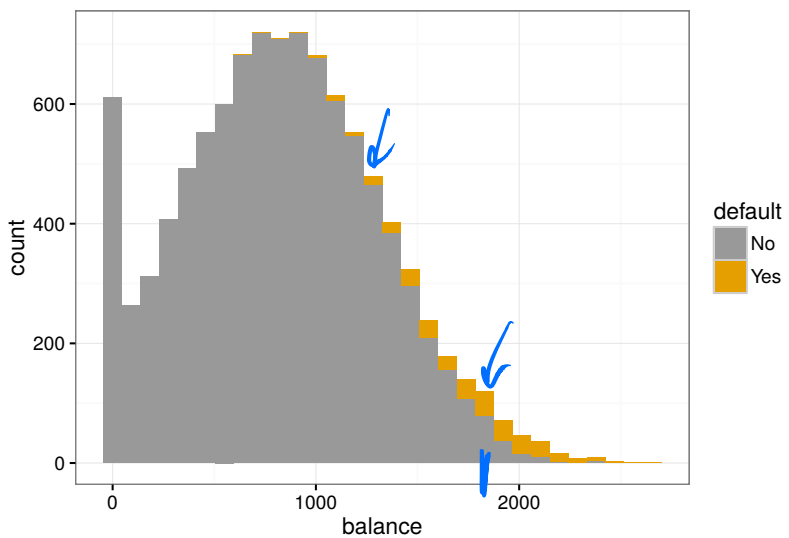
$P(Y=1 \mid X=x)$



If we have many values of x, we can't directly look at $P(Y = k|X = x)$ for each x. We need some way to pool information from *similar* points.

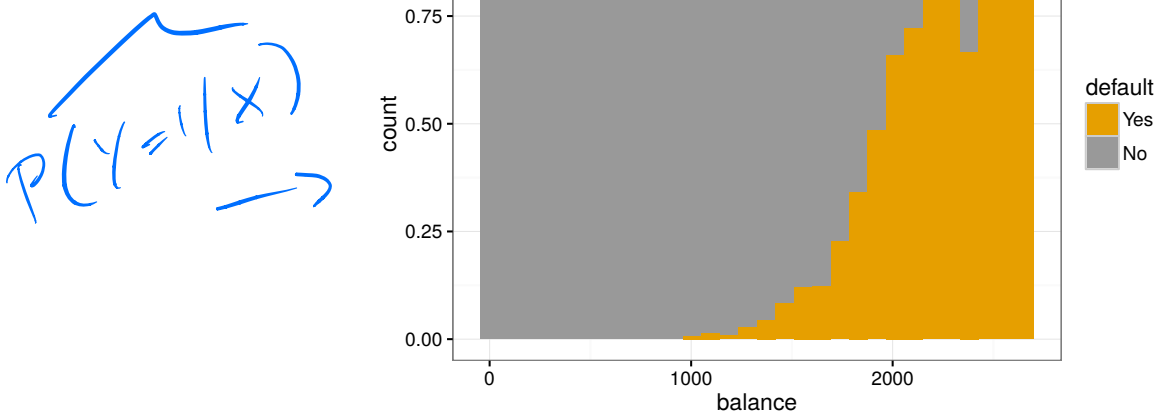
This should remind you of regression.

How should we model $P(Y = 1|X = x)$ for a continuous X?



A histogram is not quite the right picture for what happens conditional on X, but we can start to see that most individuals do not default, and that large balance seems to be related to default.

This is a conditional density plot. We look at the *probability of default* within each bin.

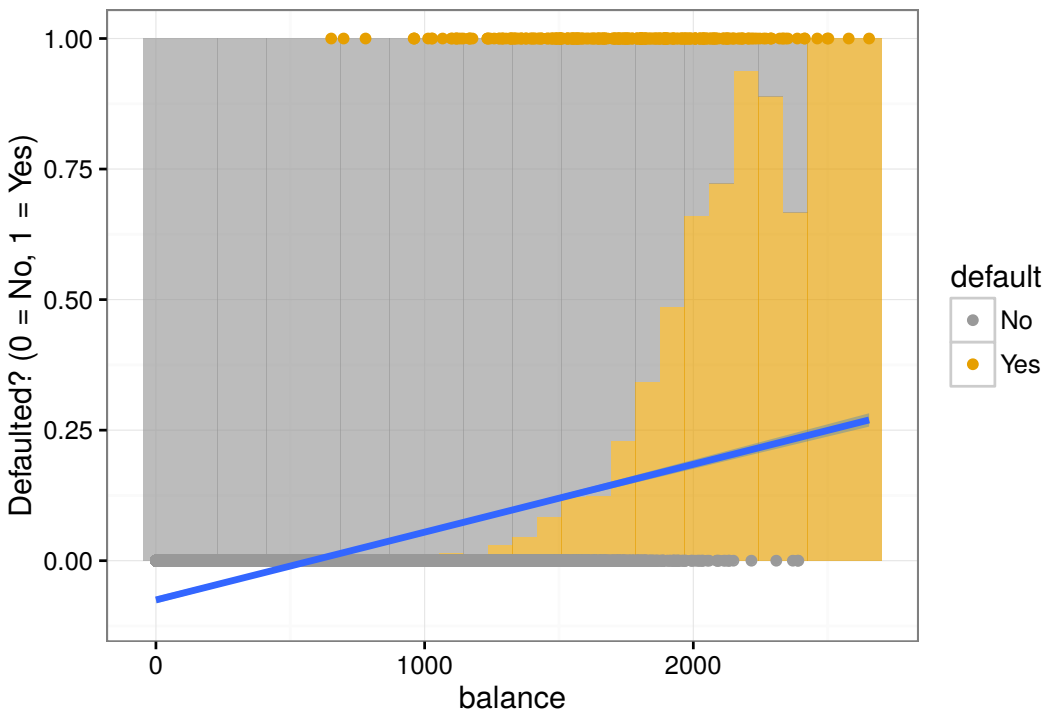


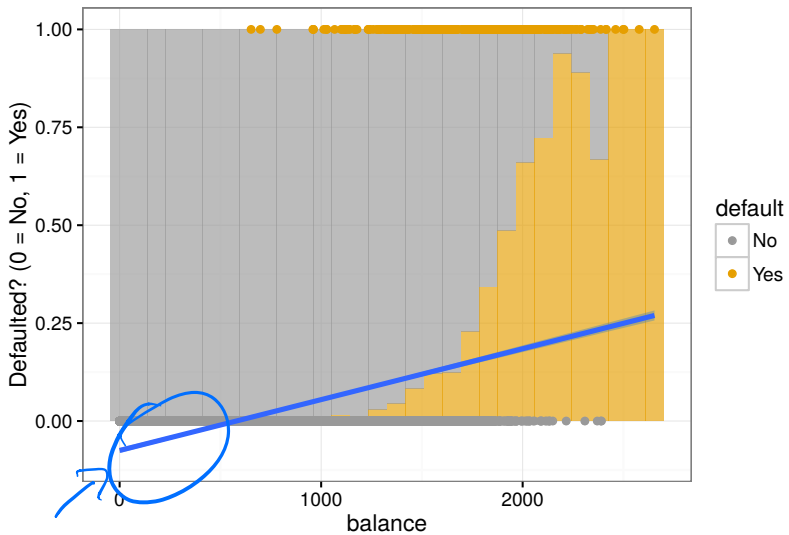
This is a (binned) plot of $P(Y = 1|X = x)$! This is clearly a natural way to think about classification. If I know which bin you are in (X), I can look at how likely you are to default.

How should I build a model of this?

We could try a linear model:

$$P(Y = 1|x = x) = \beta_0 + \beta_1 x$$





Are you happy?

- Predictions outside $[0, 1]$ don't really make sense.
- Interpretations of β are strange.
- Least squares doesn't really make sense for estimation.

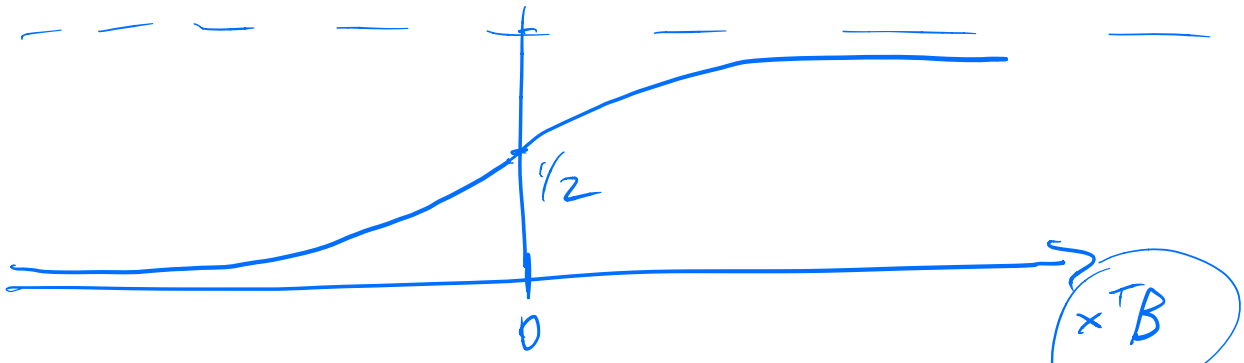
Linear regression doesn't work very well for estimating $P(Y = 1|X = x)$, since

- It doesn't make sense to extrapolate outside of $[0, 1]$.
- Least squares is an odd way to approximate probabilities.

We still like the idea of forming linear functions of our data, $\beta_0 + x_1\beta_1$ (who doesn't?).

We want a way to squish that linear function back into $[0, 1]$:

$$P(Y = 1|X = x) = \text{squish}(\beta_0 + \beta_1 x_1)$$



Logistic regression

In logistic regression, we model

$$\log\left\{\frac{\mathrm{P}(Y=1|X=x)}{\mathrm{P}(Y=0|X=x)}\right\} = \beta_0 + \beta_1 x + \cdots = \beta^T x$$

for some unknown $\beta \in \mathbb{R}^p$, which we will estimate directly

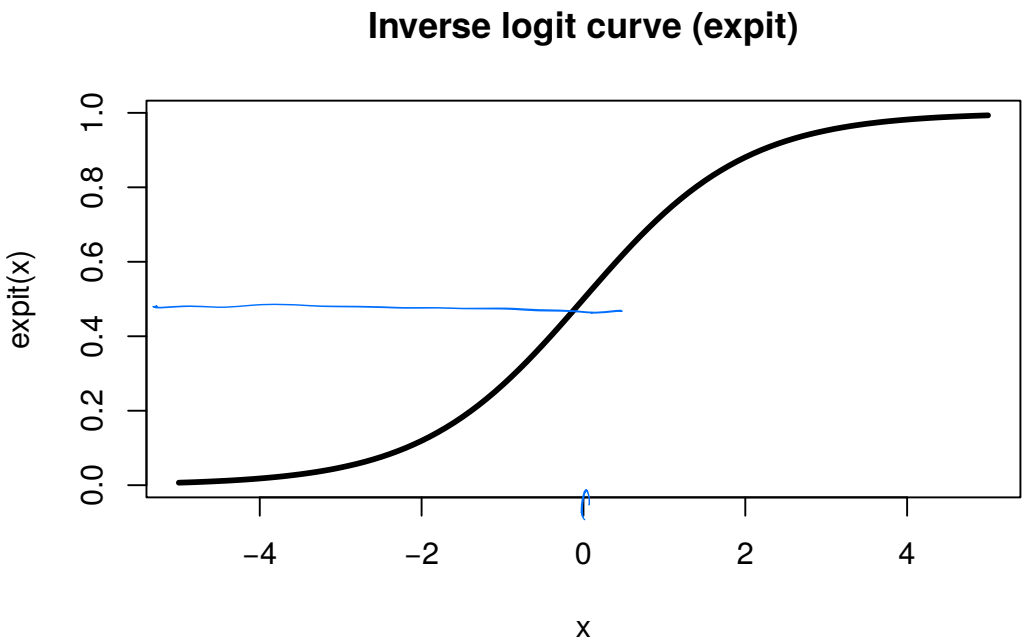
Note that $P(Y=0|X=x) = 1 - P(Y=1|X=x)$, and

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta^T x \quad \Leftrightarrow \quad \frac{p}{1-p} = e^{\beta^T x} \Leftrightarrow p = (1-p)e^{\beta^T x}$$

$$\Leftrightarrow \quad p = \frac{e^{\beta^T x}}{1+e^{x^T\beta}} = \frac{1}{1+e^{-x^T\beta}}$$

So our model is equivalent to

$$\mathrm{P}(Y=1|X=x) = \frac{\exp(\beta^T x)}{1+\exp(\beta^T x)} = \exp it(x^T\beta)$$



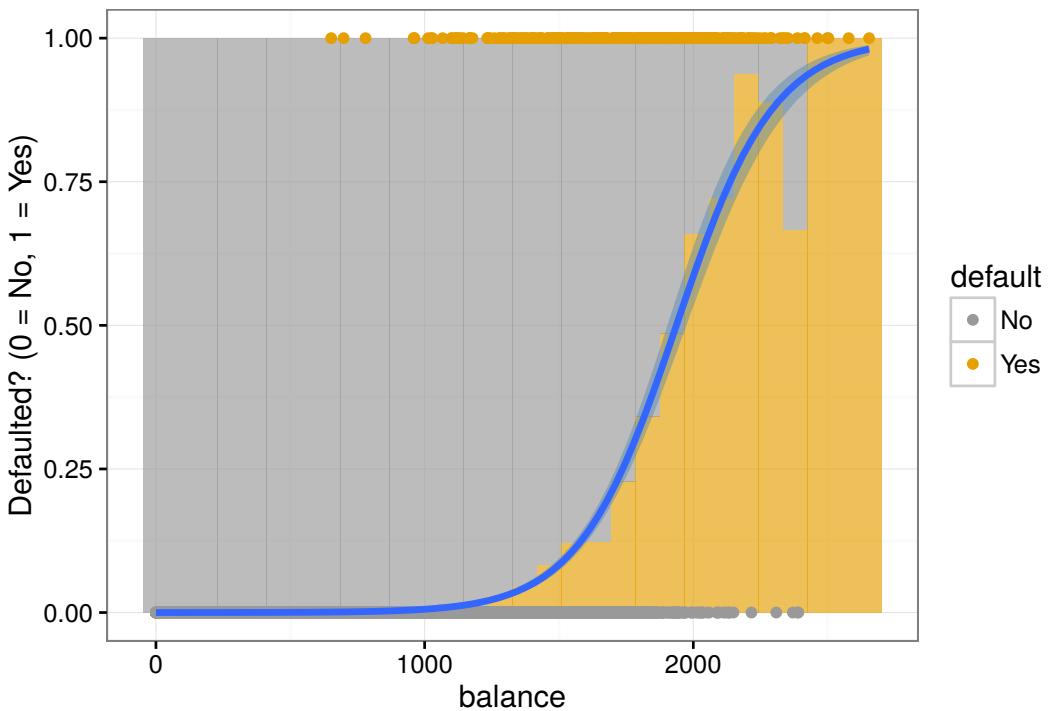
The function

$$\text{logit}^{-1}(z) = \text{expit}(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

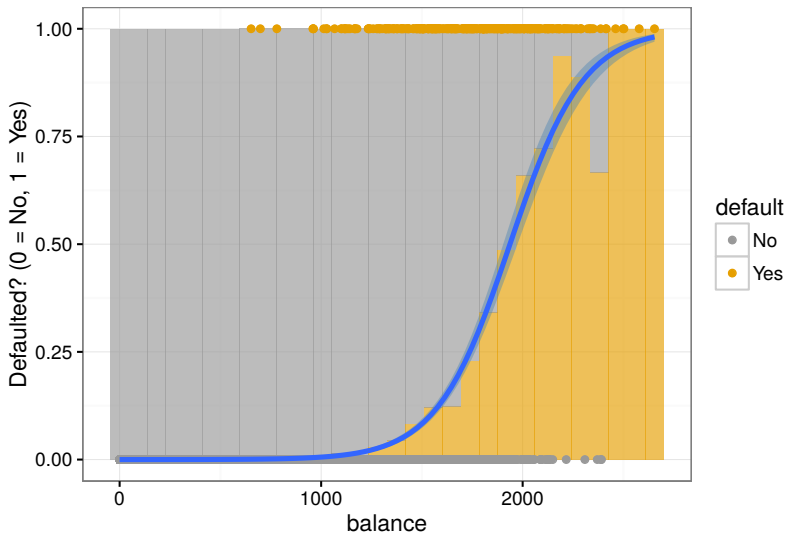
is our desired "squishing" function, transforming real numbers into $[0, 1]$.

Logistic regression and the Default data

The logistic fit gives a much more reasonable estimate of $P(Y = 1|X = x)$!



Logistic regression and the Default data



$$\log \frac{P(\text{Default}|\text{Balance})}{1 - P(\text{Default}|\text{Balance})} = \beta_0 + \beta_1 \cdot \text{Balance}$$

Logistic regression: Estimated probabilities

Once we have estimated $\hat{\beta}_0, \hat{\beta}_1$, we can estimate conditonal probabilities:

$$\mathrm{P}(Y = 1|X = x) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 x)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 x)}$$

For a balance of \$1000 or \$2000,

$$\hat{p}(1000) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 \cdot 1000)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 \cdot 1000)} = 0.00576$$

$$\hat{p}(2000) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 \cdot 2000)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 \cdot 2000)} = 0.586$$

Interpretation of logistic regression coefficients

We start to see that the coefficients are *interpretable*.

We have modeled

$$\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)} = \beta_0 + \beta_1 x_1$$

The left side, $\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)}$, is called the *log-odds* that $Y = 1$.

This means that the odds that $Y = 1$,

$$\frac{P(Y=1|X=x)}{P(Y=0|X=x)} = e^{\beta_0 + \beta_1 x_1} = e^{\beta_0} e^{\beta_1 x_1}$$

Increasing x_1 by one unit increases the estimated odds that $Y = 1$ by e^{β_1}.

Classification by logistic regression

Suppose that we fit a logistic regression, estimating $\hat{\beta}$. How do we classify?

Recall that our optimal classifier chooses

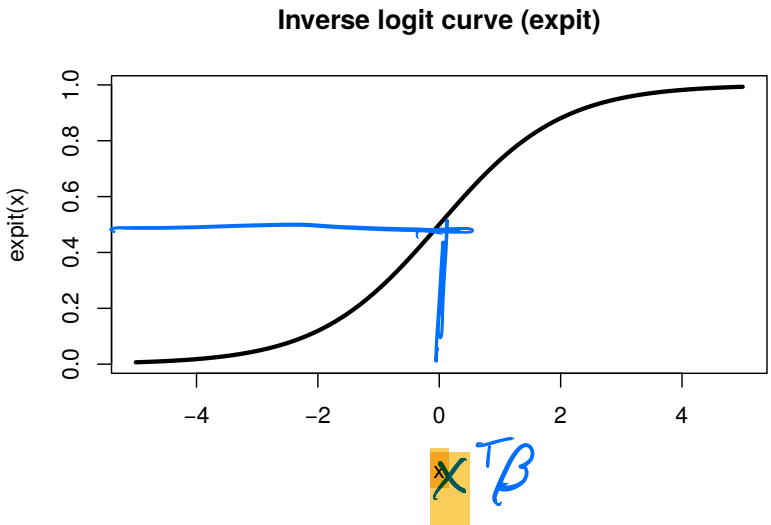
$$\underset{k}{\operatorname{argmax}}\, P(Y = k|X = x)$$

to minimize 0-1 loss.

Logistic regression gives us an estimate of $P(Y = k|X = x)$! We can just pick the category with the biggest value.

$$\hat{f}(x) = \begin{cases} 1 & \text{if } \frac{\exp(x^T\hat{\beta})}{1+\exp(x^T\hat{\beta})} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

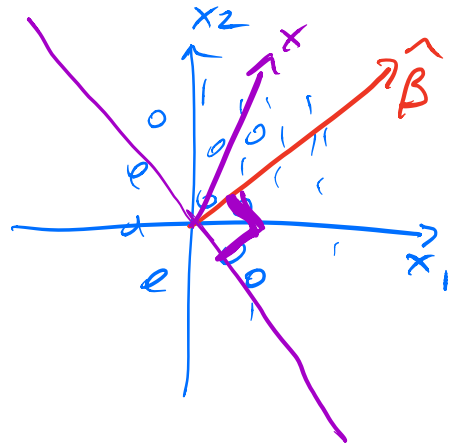
Classification by logistic regression



Remember that logit and expit are monotonically increasing! This gives us a much simpler rule!

$$\frac{\exp(x^T\hat{\beta})}{1+\exp(x^T\hat{\beta})} > 0.5 \quad \Leftrightarrow \quad x^T\hat{\beta} > 0$$

Classification by logistic regression



This gives our final decision rule

$$\hat{f}(x) = \begin{cases} 1 & \text{if } x^T\hat{\beta} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Therefore the decision boundary between classes 1 and 0 is the set of all $x \in \mathbb{R}^p$ such that

$$x^T\beta = 0$$

This is a point in $\mathbb{R}^1$ or a line in $\mathbb{R}^2$.

Estimating logistic regression coefficients

To actually estimate the $\hat{\beta}$, we just use maximum likelihood!

Suppose that we are given an i.i.d. sample (x_i, y_i), $i = 1, \ldots n$. Here y_i denotes the class $\in \{0, 1\}$ of the ith observation. Then

$$\mathcal{L}(\beta) = \prod_{i=1}^{n} \mathrm{P}(Y = y_i | X = x_i)$$

the likelihood of these n observations, so the log likelihood is

$$\ell(\beta) = \sum_{i=1}^{n} \log \mathrm{P}(Y = y_i | X = x_i)$$

We just plug in our logistic model for these probabilities and optimize.

$p(x_i) = P(Y=1 | X=x)$

The log likelihood can be written as

$$\ell(\beta) = \sum_{i=1}^{n} \log \mathrm{P}(C = y_i | X = x_i) = \sum_{i=1}^{n} \log\left[p(x_i)^{y_i} (1-p(x_i))^{1-y_i} \right]$$

$$= \sum_{i=1}^{n} y_i \log p_i + (1-y_i) \log(1-p_i)$$

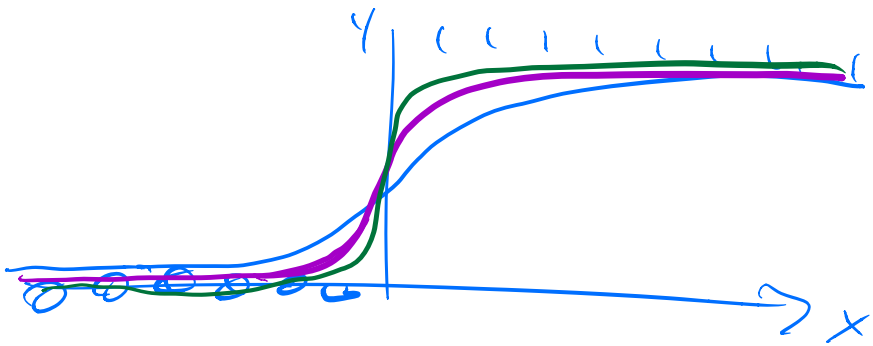
$$= \sum_{i=1}^{n} \left\{ y_i \cdot (\beta^T x_i) - \log\left(1 + \exp(\beta^T x_i)\right) \right\}$$

The coefficients are estimated by maximizing the likelihood,

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmax}} \sum_{i=1}^{n} \left\{ y_i \cdot (\beta^T x_i) - \log\left(1 + \exp(\beta^T x_i)\right) \right\}$$

replace with $\sum_j \hat{f}_j(x_i)$

What if I can predict perfectly?



What if I have many variables?

Suppose that the number of variables is large. We would expect the variance of our estimated $\hat{\beta}$ to grow!

What could we do to reduce the variance?

Ridge. $\max_{\beta}$ log Likelihood $- \lambda \|\beta\|_2$

What if we also wanted to pick a sparse, interpretable linear model?

Lasso $\max_{\beta}$ log lik $- \lambda \|\beta\|_1$

glmnet: family = "binomial"

What if we need a more flexible model?

What if we don't really believe that the log odds increase linearly in each of our variables? What if we don't know how to transform our variables?

generalized additive models

Classification so far

Focus on binary classification,

- $Y = 1$ if an event of interest happened
- $Y = 0$ if it did not happen

Most of our classifiers will give us a guess $\hat{p}(x) = \hat{\mathbb{P}}(Y = 1|X = x)$.
This is an estimate of the true $p(x) = \mathbb{P}(Y = 1|X = x)$.

We saw that to minimize average misclassification error, $\mathbb{E}\mathbf{1}_{f(X)\neq Y}$, you should choose $f(x) = \mathbf{1}_{p(x)>1/2}$.

Let's think about this more carefully.

Misclassification rate

Suppose that I tell you my classifier has 97% accuracy. Is this good?

Misclassification rate

Suppose that I tell you my classifier has 97% accuracy. Is this good?

It depends! What if I am predicting lottery wins. The probability of winning may be $1/175000000$.

In that case, guessing "no" has a misclassification rate of 5×10^{-9}...

We need to consider the *base rate*: the probability of each class given no further information.

Confusion matrix: Example

For the homework problem (to be), $Y = 1$ if an individual responds to marketing. The classification rule $\hat{p}(x) > 0.5$ gives a confusion matrix similar to:

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	2
Guess $Y = 0$	1083	7957